

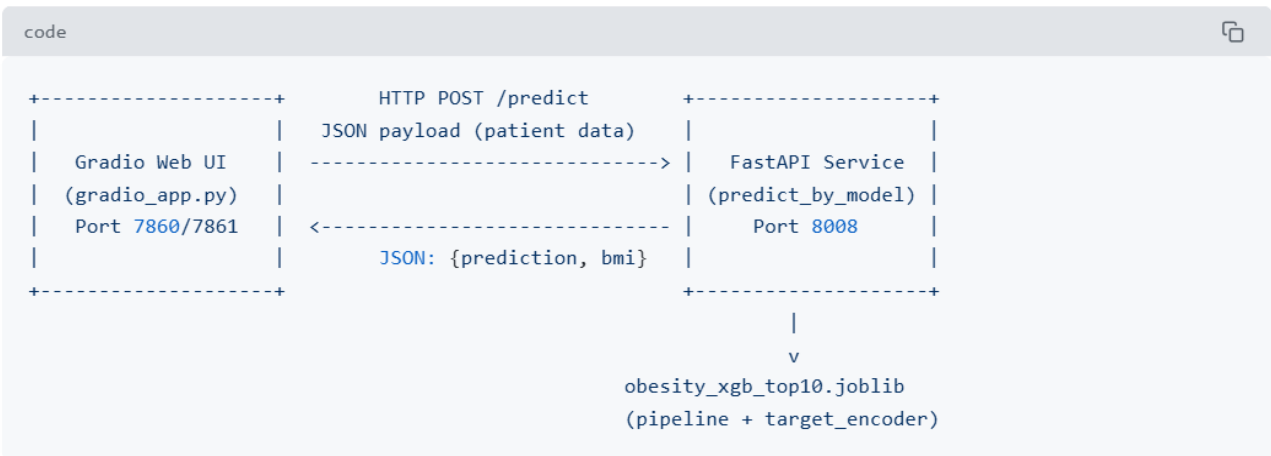
Obesity Prediction App — Full Deployment Documentation

This document describes the prediction service end-to-end:

1. The **prediction API** (`predict_by_model.py`) built with FastAPI.
2. The **Gradio UI** (`gradio_app.py`) that talks to the API.
3. The **two Dockerfiles** used to build container images for each service.
4. All Docker commands needed to build, run, connect, and deploy the two containers.

The model artifact used at inference time is `obesity_xgb_top10.joblib`, an XGBoost pipeline trained on the top-10 features derived from `train.csv` (see `requirements.txt` for the training environment).

1. Architecture Overview



Two independent containers are deployed:

Container	Image (suggested tag)	Internal port	Source files
API	<code>obesity-api:latest</code>	<code>8008</code>	<code>Dockerfile.api</code> , <code>predict_by_model.py</code> , <code>obesity_xgb_top10.joblib</code> , <code>requirements.txt</code>
Gradio	<code>obesity-gradio:latest</code>	<code>7860</code>	<code>Dockerfile.gradio</code> , <code>gradio_app.py</code>

The Gradio container reaches the API container over the network using the URL passed in the `PREDICT_API_URL` environment variable.

2. The Prediction API — `predict_by_model.py`

2.1 Responsibilities

- Load the trained model bundle (`obesity_xgb_top10.joblib`).
- Expose a `/predict` HTTP endpoint accepting patient data.

- Compute BMI automatically when missing.
- Return the predicted obesity class label and the computed BMI as JSON.

2.2 Model Bundle Contents

`joblib.load("obesity_xgb_top10.joblib")` returns a dictionary with:

- `pipeline` — fitted scikit-learn / XGBoost pipeline (preprocessing + classifier).
- `target_encoder` — the encoder used to decode predicted class indices.
- `feature_cols` — the ordered list of the 10 features the model expects.

The 10 expected features (fallback constant):

code



```
Age, FCVC, NCP, CH2O, FAF, TUE, BMI,
family_history_with_overweight, FAVC, CALC
```

2.3 Request Schema (PatientData)

The API accepts a JSON body that uses **human-readable names**. Internally it remaps them to the model feature names.

API field name	Type	Mapped to model feature	Notes
Height	float	(used to compute BMI)	metres
Weight	float	(used to compute BMI)	kg
Age	float	Age	years
Vegetable_consumption_frequency	float	FCVC	1–3
main_meals_count	float	NCP	1–4
water_consumption	float	CH2O	1–3
Physical_activity_frequency	float	FAF	0–3
Tech_device_time	float	TUE	0–3
family_history_with_overweight	string	family_history_with_overweight	"yes" / "no"
High_calorie_food	string	FAVC	"yes" / "no"
Alcohol_consumption	string	CALC	"no", "Sometimes", "Frequently", "Always"

BMI is calculated as `Weight / (Height ** 2)` if missing.

2.4 Endpoints

Method	Path	Purpose
GET	/healthy	Health check. Returns <code>{"status": "ok", "docs":</code>

		"/docs"}.
GET	/docs	Auto-generated Swagger UI (provided by FastAPI).

2.5 Label Map

The numeric output of the pipeline is decoded with:

code



```
0 -> Underweight
1 -> Normal
2 -> Overweight
3 -> Obesity
```

If the pipeline already returns string labels, they are returned as-is.

2.6 Example Request / Response

Request:

json



```
POST /predict
{
  "Height": 1.75,
  "Weight": 82.0,
  "Age": 28,
  "Vegetable_consumption_frequency": 2.0,
  "main_meals_count": 3.0,
  "water_consumption": 2.0,
  "Physical_activity_frequency": 1.0,
  "Tech_device_time": 1.0,
  "family_history_with_overweight": "yes",
  "High_calorie_food": "no",
  "Alcohol_consumption": "Sometimes"
}
```

Response:

json



```
{
  "prediction": "Overweight",
  "bmi": 26.78
}
```

2.7 Local Run (without Docker)

bash



```
pip install -r requirements.txt
pip install fastapi uvicorn pydantic "scikit-learn==1.6.1"
python predict_by_model.py
# Server listens on http://0.0.0.0:8008
```

3. The Gradio UI — `gradio_app.py`

3.1 Responsibilities

- Render an HTML form for the 11 user-facing inputs.
- Build a JSON payload matching the API schema.
- POST it to the URL stored in `PREDICT_API_URL`.
- Render the prediction inside a styled result card (or an error card on failure).

3.2 Configuration

The target API URL is read from an environment variable:

```
python

API_URL = os.environ.get("PREDICT_API_URL", "http://localhost:8008/predict")
```

This means **you do not rebuild the image** to change the backend — you pass `-e PREDICT_API_URL=...` at `docker run` time.

3.3 Error Handling

Three failure paths are handled and each renders a red card:

- `requests.exceptions.ConnectionError` — API unreachable.
- `requests.exceptions.HTTPError` — API responded with a non-2xx status.
- Any other exception — rendered as “Unexpected Error”.

3.4 Port Note

`demo.launch(server_name="0.0.0.0", server_port=7861)` runs the app on **7861** inside the container, but `Dockerfile.gradio` only `EXPOSE`s **7860** (the Gradio default). `EXPOSE` is informational only — it does not block traffic — but for clarity, publish the port the app actually listens on:

```
bash

# the app inside the container is on 7861
-p 7860:7861
```

Either align them (recommended: set `server_port=7860` in the code) or remember the mapping above when running. The commands in section 5 use this mapping so the published host port `7860` works without code changes.

3.5 Local Run (without Docker)

```
bash

pip install gradio requests
export PREDICT_API_URL=http://localhost:8008/predict # PowerShell: $env:PREDICT_API_URL="..."
python gradio_app.py
# UI is at http://localhost:7861
```

4. The Dockerfiles

4.1 Dockerfile.api — line-by-line

dockerfile



```
FROM python:3.11-slim
```

Use the slim Python 3.11 base image. `slim` keeps the image small (no compilers, no extra tooling), which is fine because all our dependencies are pure-Python wheels or have manylinux wheels.

dockerfile



```
WORKDIR /app
```

All subsequent `COPY` and `CMD` operations happen inside `/app`. This becomes the container's working directory at runtime.

dockerfile



```
COPY requirements.txt .
```

Copy only the requirements file first. This is the **layer-caching trick**: as long as `requirements.txt` does not change, Docker reuses the cached pip-install layer when you rebuild after editing source code.

dockerfile



```
RUN pip install --no-cache-dir -r requirements.txt \  
&& pip install --no-cache-dir fastapi uvicorn pydantic "scikit-learn==1.6.1"
```

Install all training-stack dependencies from `requirements.txt`, then add the serving-stack dependencies (`fastapi`, `uvicorn`, `pydantic`) and **pin** `scikit-learn==1.6.1`. The pin is critical: the joblib bundle was created with that exact version, and unpickling a scikit-learn pipeline with a different minor version can fail or silently misbehave. `--no-cache-dir` avoids leaving the pip download cache inside the image.

dockerfile



```
COPY predict_by_model.py .  
COPY obesity_xgb_top10.joblib .
```

Copy the application code and the trained model artifact into `/app`.

dockerfile



```
EXPOSE 8008
```

Documents that the service listens on port 8008. Does not actually publish the port — that happens with `docker run -p ...`.

dockerfile



```
CMD ["uvicorn", "predict_by_model:app", "--host", "0.0.0.0", "--port", "8008"]
```

Start the API server. `--host 0.0.0.0` makes it reachable from outside the container; `predict_by_model:app` references the `app` object inside `predict_by_model.py`. `CMD` uses the exec-form list so signals (`SIGTERM` on `docker stop`) reach uvicorn directly.

4.2 Dockerfile.gradio — line-by-line

```
dockerfile
```



```
FROM python:3.11-slim
```

Same slim base — kept consistent with the API image.

```
dockerfile
```



```
WORKDIR /app
```

Container working directory.

```
dockerfile
```



```
RUN pip install --no-cache-dir gradio requests
```

The Gradio app only needs `gradio` (for the UI) and `requests` (to call the API). We deliberately do **not** install `pandas`, `scikit-learn`, or the model — keeping the UI image small and free of ML dependencies.

```
dockerfile
```



```
COPY gradio_app.py .
```

Copy the Gradio app source.

```
dockerfile
```



```
ENV PREDICT_API_URL="http://192.168.1.118:8008/predict"
```

Default backend URL baked into the image. **Override at runtime** with `docker run -e PREDICT_API_URL=...`. This default exists only as a sensible fallback; in production you should always pass an explicit value.

```
dockerfile
```



```
EXPOSE 7860
```

Documents the Gradio default port. Note the mismatch with `server_port=7861` in `gradio_app.py` (see §3.4).

```
dockerfile
```



```
CMD ["python", "gradio_app.py"]
```

Start the Gradio app.

5. Docker Commands — Build, Run, Inspect, Tear Down

The commands below assume your shell's current directory contains both Dockerfiles and source files.

5.1 Build the images

```
bash

# Build the API image
docker build -t obesity-api:latest -f Dockerfile.api .

# Build the Gradio image
docker build -t obesity-gradio:latest -f Dockerfile.gradio .
```

Flag meanings:

- `-t obesity-api:latest` — tag the resulting image.
- `-f Dockerfile.api` — use this Dockerfile instead of the default `Dockerfile`.
- `.` — the build context (current directory). Files outside this directory cannot be `COPY`-ed.

To force a clean rebuild (ignore cached layers):

```
bash

docker build --no-cache -t obesity-api:latest -f Dockerfile.api .
```

5.2 Run the API container

Simplest run, publishing the port to the host:

```
bash

docker run -d --name obesity-api -p 8008:8008 obesity-api:latest
```

Flag meanings:

- `-d` — detached (run in the background).
- `--name obesity-api` — give the container a stable name (used later for logs/stop/network).
- `-p 8008:8008` — publish container port 8008 to host port 8008.

Verify:

```
bash

curl http://localhost:8008/healthy
# {"status":"ok","docs":"/docs"}
```

Open Swagger UI in a browser: <http://localhost:8008/docs>

5.3 Run the Gradio container — option A: shared host network (simplest on Linux)

If both containers run on the same Linux host, you can let Gradio talk to the API via `host.docker.internal` (Docker Desktop on Windows/macOS) or directly via the host IP:

bash



```
docker run -d --name obesity-gradio \
  -p 7860:7861 \
  -e PREDICT_API_URL=http://host.docker.internal:8008/predict \
  obesity-gradio:latest
```

Note the port mapping `-p 7860:7861` — host port 7860 → container port 7861 (the port the Gradio app actually listens on; see §3.4). Open <http://localhost:7860> in a browser.

5.4 Run both containers — option B: user-defined bridge network (recommended)

This is the cleanest setup: both containers join a private network and the Gradio container reaches the API by container name.

bash



```
# 1. Create a dedicated bridge network
docker network create obesity-net

# 2. Run the API on that network
docker run -d --name obesity-api \
  --network obesity-net \
  -p 8008:8008 \
  obesity-api:latest

# 3. Run Gradio on the same network and point it at the API by name
docker run -d --name obesity-gradio \
  --network obesity-net \
  -p 7860:7861 \
  -e PREDICT_API_URL=http://obesity-api:8008/predict \
  obesity-gradio:latest
```

Inside `obesity-net`, Docker's embedded DNS resolves `obesity-api` to the API container's IP. This works the same on any host and is what you want in production.

5.5 Inspect, debug, and stream logs

bash



```
docker ps                # list running containers
docker logs -f obesity-api # tail API logs
docker logs -f obesity-gradio # tail Gradio logs
docker exec -it obesity-api sh # shell inside the API container
docker inspect obesity-api  # full JSON dump (env, mounts, network)
```

5.6 Stop, restart, remove

bash



```
docker stop obesity-gradio obesity-api
docker start obesity-api obesity-gradio
docker rm -f obesity-gradio obesity-api # remove containers
docker network rm obesity-net           # remove the bridge network
docker rmi obesity-api:latest obesity-gradio:latest # remove images
```


5.7 Push to a registry (optional)

bash



```
# Tag for your registry
docker tag obesity-api:latest <registry>/<namespace>/obesity-api:1.0.0
docker tag obesity-gradio:latest <registry>/<namespace>/obesity-gradio:1.0.0

# Push
docker login <registry>
docker push <registry>/<namespace>/obesity-api:1.0.0
docker push <registry>/<namespace>/obesity-gradio:1.0.0
```

6. Optional `docker-compose.yml`

For one-command deployment, the same two services can be expressed as Compose. Save this as `docker-compose.yml` and run `docker compose up -d`.

yml



```
version: "3.9"

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    image: obesity-api:latest
    container_name: obesity-api
    ports:
      - "8008:8008"
    restart: unless-stopped

  gradio:
    build:
      context: .
      dockerfile: Dockerfile.gradio
    image: obesity-gradio:latest
    container_name: obesity-gradio
    depends_on:
      - api
    environment:
      PREDICT_API_URL: "http://api:8008/predict"
    ports:
      - "7860:7861"
    restart: unless-stopped
```

Commands:

bash



```
docker compose build      # build both images
docker compose up -d      # start both containers
docker compose logs -f    # follow logs
docker compose down       # stop and remove
```

7. End-to-End Smoke Test

After running both containers:

```
bash

# 1. Health check on the API
curl http://localhost:8008/healthy

# 2. Direct prediction call (bypass the UI)
curl -X POST http://localhost:8008/predict \
  -H "Content-Type: application/json" \
  -d '{
    "Height": 1.75,
    "Weight": 82,
    "Age": 28,
    "Vegetable_consumption_frequency": 2.0,
    "main_meals_count": 3.0,
    "water_consumption": 2.0,
    "Physical_activity_frequency": 1.0,
    "Tech_device_time": 1.0,
    "family_history_with_overweight": "yes",
    "High_calorie_food": "no",
    "Alcohol_consumption": "Sometimes"
  }'

# 3. Open the Gradio UI in a browser
#   http://localhost:7860
```

The UI should fill in the form, hit the API, and render the predicted obesity class in the result card.

8. Files at a Glance

File	Role
Dockerfile.api	Builds the FastAPI image
Dockerfile.gradio	Builds the Gradio image
predict_by_model.py	FastAPI service — loads model, exposes /predict
gradio_app.py	Gradio UI — form + HTTP client
obesity_xgb_top10.joblib	Trained pipeline + target encoder
requirements.txt	Python deps for training & API runtime